

黑白棋项目代码简介

1 GUI

1.1 Swing包

在Demo中使用的库是Java Swing库。

Swing 库的主要功能和特点：

- **轻量级组件：** Swing 提供了一些比 AWT 更加灵活和功能丰富的轻量级组件，例如 `JFrame`、`JButton`、`JPanel` 等。
- **可定制的外观：** Swing 组件的外观可以通过 "Look and Feel" 改变，因此它们能够模仿不同平台上的用户界面风格。
- **丰富的布局管理器：** Swing 提供了多种布局管理器（例如 `BorderLayout`、`GridLayout`、`FlowLayout`），以便组织和排列不同的组件。

在代码中，主要涉及到的 Swing 组件有：

- `JFrame`：顶层容器，用来创建主应用程序窗口。
- `JButton`：表示一个按钮，用于接收用户点击事件。
- `JPanel`：容器组件，可以用来组织其他组件，例如按钮、标签等。

通过 Swing，你可以创建直观且用户友好的图形用户界面，正如你项目中用于构建黑白棋的交互界面。

1.2 GUI主要代码

```
public class ReversiGUI extends JFrame {
    private JButton[][] boardButtons;
    private ReversiGame game;
    private int boardSize;
    private int maxTimePerMove;

    public ReversiGUI(int size, ReversiGame game) {
        this.boardSize = size;
        this.game = game;
        boardButtons = new JButton[boardSize][boardSize];
        initializeBoard();
    }

    private void initializeBoard() {
        setTitle("Reversi Game");
        setSize(800, 900);
        setLayout(new BorderLayout());
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        String input = JOptionPane.showInputDialog(this, "Enter maximum time per
move (in seconds):");
        try {
            maxTimePerMove = Integer.parseInt(input);
        } catch (NumberFormatException e) {
            maxTimePerMove = 30; // Default to 30 seconds if input is invalid
        }
    }
}
```

```

JPanel boardPanel = new JPanel(new GridLayout(boardSize, boardSize));

for (int i = 0; i < boardSize; i++) {
    for (int j = 0; j < boardSize; j++) {
        boardButtons[i][j] = new JButton();
        boardButtons[i][j].setBackground(Color.GREEN);
        int row = i;
        int col = j;
        boardButtons[i][j].addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                handleBoardClick(row, col);
            }
        });
        boardPanel.add(boardButtons[i][j]);
    }
}

JButton saveButton = new JButton("Save Game");
saveButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(ReversiGUI.this, "Game saved!");
    }
});

JButton loadButton = new JButton("Load Game");
loadButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(ReversiGUI.this, "Game loaded!");
    }
});

JPanel buttonPanel = new JPanel(new FlowLayout());
buttonPanel.add(saveButton);
buttonPanel.add(loadButton);

add(boardPanel, BorderLayout.CENTER);
add(buttonPanel, BorderLayout.SOUTH);
updateBoard(game.getBoardState());
}

private void handleBoardClick(int row, int col) {
    if (game.makeMove(row, col)) {
        updateBoard(game.getBoardState());
    } else {
        JOptionPane.showMessageDialog(this, "Invalid move!", "Error",
JOptionPane.ERROR_MESSAGE);
    }
}

public void updateBoard(int[][] boardState) {
    for (int i = 0; i < boardSize; i++) {
        for (int j = 0; j < boardSize; j++) {

```

```

        if (boardState[i][j] == 1) {
            boardButtons[i][j].setBackground(Color.BLACK);
        } else if (boardState[i][j] == 2) {
            boardButtons[i][j].setBackground(Color.WHITE);
        } else {
            boardButtons[i][j].setBackground(Color.GREEN);
        }
    }
}
}
}
}

```

1.3 页面布局

在 Java GUI 开发中，`BorderLayout`、`GridLayout` 和 `FlowLayout` 是常用的布局管理器，它们各自有不同的特点和使用场景。以下是对它们的详细介绍：

1. BorderLayout

- **定义：**`BorderLayout` 是 Java Swing 中最常见的布局管理器之一，它将容器分为五个区域：北（NORTH）、南（SOUTH）、东（EAST）、西（WEST）和中（CENTER）。
- **使用场景：**适合用于需要将界面划分为几大部分的场景。例如，你可以将按钮放置在南部，主内容放在中部，工具栏放在北部等。

- **特点**

:

- 每个区域只能放置一个组件，组件会占据其所在的整个区域。
- 如果只添加中部区域的组件，它将占据整个容器。
- 如果多个组件放在同一位置，则会覆盖掉前面的组件。

代码示例：

```

setLayout(new BorderLayout());
add(boardPanel, BorderLayout.CENTER); // 棋盘面板在中部
add(buttonPanel, BorderLayout.SOUTH); // 按钮面板在底部

```

2. GridLayout

- **定义：**`GridLayout` 将容器划分为指定数量的行和列，并将组件均匀地分布到这些网格中。
- **使用场景：**适用于需要创建规则网格布局的场景，例如棋盘、计算器按键等。
- **特点：**
 - 每个网格大小相等，组件会被均匀分布在网格中。
 - 无法控制每个组件的大小，每个组件都占据相同的空间。
 - 在 `GridLayout(rows, cols)` 中，可以指定行数和列数，组件按照从左到右、从上到下的顺序排列。

代码示例：

```

JPanel boardPanel = new JPanel(new GridLayout(boardSize, boardSize)); // 例如
                          创建一个8x8的棋盘

```

3. FlowLayout

- **定义：** `FlowLayout` 是默认的布局管理器，它按组件添加的顺序将它们从左到右、从上到下排列，类似于文本的流式布局。
- **使用场景：** 适合用于需要自然排列组件的场景，例如按钮区域、工具栏等。
- **特点：**
 - 组件会从左到右、从上到下进行排列，直到容器的边界为止，然后自动换行。
 - 可以设置对齐方式，例如左对齐（`FlowLayout.LEFT`）、右对齐（`FlowLayout.RIGHT`）、居中（`FlowLayout.CENTER`）等。
 - 对于需要一个简单的线性布局而无需精确控制位置和大小场景非常适合。

代码示例：

```
JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER)); // 按钮会在中间排列
```

总结：

- **BorderLayout：** 适用于将组件放在不同区域（如顶部、底部、中部）的场景。适合用来构建主界面的大框架。
- **GridLayout：** 适合需要均匀分布组件的场景，例如棋盘或计算器按键。每个组件的大小相同。
- **FlowLayout：** 适合线性排列组件，通常用于按钮面板、工具栏等，需要自然排列而不关心精确位置和大小情况。

不同的布局管理器各有优势，选择时应根据界面设计的具体需求来确定。通过组合使用这些布局管理器，可以设计出功能丰富且美观的用户界面。

1.4 界面按钮

```
JPanel boardPanel = new JPanel(new GridLayout(boardSize, boardSize));
```

- 这段代码创建了一个名为 `boardPanel` 的面板，它使用 `GridLayout` 布局管理器来管理子组件的布局。
- `GridLayout(boardSize, boardSize)` 创建一个大小为 `boardSize x boardSize` 的网格布局，这里用于表示 8x8 的棋盘。

```
for (int i = 0; i < boardSize; i++) {  
    for (int j = 0; j < boardSize; j++) {  
        boardButtons[i][j] = new JButton();  
        boardButtons[i][j].setBackground(Color.GRAY);  
        int row = i;  
        int col = j;
```

- 双重循环用于初始化棋盘上的每一个按钮，按钮保存在 `boardButtons` 二维数组中。
- `new JButton()` 创建了一个按钮对象，这些按钮代表棋盘上的每个格子。
- `setBackground(Color.GRAY)` 设置按钮的背景颜色为灰色。
- `int row = i; int col = j;` 是为了保存按钮的位置，因为在后面的匿名类中需要使用这些坐标。

- 在 Java 中，匿名内部类（如 `ActionListener`）可以访问所在作用域中的**局部变量**，但这些局部变量必须是**final**或**有效 final**的。代码中用来存储 `row` 和 `col` 的局部变量 `int row = i; int col = j;`，实际上是为了让匿名内部类在按钮点击时能够知道它代表棋盘中的哪一个位置。

这是因为在嵌套的匿名类中直接使用 `i` 和 `j` 会引发问题。如果 `i` 和 `j` 在循环中继续递增，而匿名类中的按钮监听器异步触发时，将无法保证这些变量的值始终指向正确的按钮。因此，使用 `row` 和 `col` 来保存当前循环的值，这些变量保持固定且不会改变，从而使得每个按钮都能正确传递它所对应的棋盘位置。

```
boardButtons[i][j].addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        handleBoardClick(row, col);
    }
});
```

- `addActionListener(new ActionListener())` 为每个按钮添加一个动作监听器，用于响应用户的点击。
 - 当玩家点击某个按钮时，会触发该按钮的 `ActionListener`，从而调用 `actionPerformed` 方法。
 - `actionPerformed` 方法内部调用了 `handleBoardClick(row, col)`，并将 `row` 和 `col` 传递给它。
 - `handleBoardClick(row, col)` 是你定义的一个方法，用于根据玩家的选择来更新棋盘状态，比如判断该位置是否可以落子，进行翻转等操作。
- `@Override` 表示这个方法是对父类或接口中定义的 `actionPerformed` 方法的重写，目的是确保编写的方法符合父类或接口的要求。
- `actionPerformed(ActionEvent e)` 方法定义了当按钮被点击时执行的操作。这里调用了 `handleBoardClick(row, col)` 方法，将当前按钮的行列信息传递给处理函数。

重点解释

1. `actionPerformed` 方法：

- `actionPerformed` 是 `ActionListener` 接口中的一个抽象方法，用于处理按钮被点击时的事件。
- 在这里，`actionPerformed` 方法被重写，用于调用 `handleBoardClick(row, col)` 方法，传递当前按钮的位置，从而实现棋盘上的落子操作。

2. `@Override` 注解：

- `@Override` 是 Java 的一种注解，用来表示当前的方法是对父类或接口中某个方法的重写。
- 它的作用是提高代码的可读性，并且帮助编译器进行检查，确保重写的方法的签名正确。如果方法签名有误，编译器会提示错误。这对于防止由于拼写错误或方法参数不匹配而引发的问题非常有用。

1.5 额外的按钮以及整体布局

创建按钮并添加动作监听器：

```
JButton saveButton = new JButton("Save Game");
saveButton.addActionListener(new ActionListener() {
```

```

@Override
public void actionPerformed(ActionEvent e) {
    JOptionPane.showMessageDialog(ReversiGUI.this, "Game saved!");
}
});

JButton loadButton = new JButton("Load Game");
loadButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(ReversiGUI.this, "Game loaded!");
    }
});

```

- `JButton saveButton = new JButton("Save Game");` 创建了一个按钮，按钮上显示的文本为 "Save Game"。
- `saveButton.addActionListener(new ActionListener() {...})` 为 `saveButton` 添加了一个动作监听器。当用户点击按钮时，触发 `actionPerformed` 方法。
- 在 `actionPerformed` 方法中，调用了 `JOptionPane.showMessageDialog` 方法，弹出一个消息框，通知用户 "Game saved!"。
- 对于 `loadButton`，同样的逻辑用于处理 "Load Game" 的操作。

创建面板并将按钮添加到面板中：

```

JPanel buttonPanel = new JPanel(new FlowLayout());
buttonPanel.add(saveButton);
buttonPanel.add(loadButton);

```

- `JPanel buttonPanel = new JPanel(new FlowLayout());`：创建了一个面板 `buttonPanel`，并设置布局管理器为 `FlowLayout`。
 - `FlowLayout` 是一种简单的布局管理器，它将组件按照添加的顺序从左到右进行排列，通常用于需要简单线性排列组件的场景。
- `buttonPanel.add(saveButton); buttonPanel.add(loadButton);`：将保存和加载按钮添加到面板中。`FlowLayout` 使按钮排列得很紧凑，看起来更加整齐。

将面板添加到窗口中：

```

add(boardPanel, BorderLayout.CENTER);
add(buttonPanel, BorderLayout.SOUTH);

```

- `add(boardPanel, BorderLayout.CENTER);`：将棋盘面板 `boardPanel` 添加到窗口的中心区域。
- `add(buttonPanel, BorderLayout.SOUTH);`：将按钮面板 `buttonPanel` 添加到窗口的南部（底部）。使用 `BorderLayout` 布局时，`SOUTH` 代表底部。

1.6 弹窗提示输入信息

`JOptionPane.showInputDialog` 方法：

- `JOptionPane.showInputDialog(this, "Enter maximum time per move (in seconds):");` :
 - 这一行代码会弹出一个输入对话框，提示用户输入每步操作的最大时间（单位为秒）。
 - `JOptionPane` 是 Java Swing 中提供的一种工具类，可以用来创建各种简单的对话框，如输入框、消息框、确认框等。
 - `showInputDialog` 方法用于弹出一个输入对话框，让用户输入一些文本信息。
 - `this` 参数表示对话框的父组件。在这里，它将 `ReversiGUI` 作为父组件，意味着输入框会相对于这个窗口居中显示。
 - **"Enter maximum time per move (in seconds):"** 是对话框显示的提示信息，引导用户输入操作时间。
- 这个方法会返回用户输入的文本内容作为字符串。如果用户点击 "取消" 或者关闭输入框，它将返回 `null`。

处理用户输入：

- 用户输入的内容是一个字符串 (`String input`)，需要将它转换为整数（因为我们要表示秒数）。

```
try {
    maxTimePerMove = Integer.parseInt(input);
} catch (NumberFormatException e) {
    maxTimePerMove = 30; // Default to 30 seconds if input is invalid
}
```

- **try 块**：使用 `Integer.parseInt(input)` 将用户输入的字符串转换为整数。如果用户输入的是合法的数字字符串，转换将会成功，值会赋给 `maxTimePerMove`。
- `catch (NumberFormatException e)`：如果用户输入的是非法的内容（例如字母或者其他无法转换为整数的字符串），则会抛出 `NumberFormatException` 异常。
- 在这种情况下，代码会进入 `catch` 块，并将 `maxTimePerMove` 设置为默认值 30 秒。

2 文件读取和保存

1. 使用的 Java 库

- `javax.swing`：用于 GUI 对话框操作，主要通过 `JFileChooser` 和 `JOptionPane` 实现文件选择和用户提示。
 - `JFileChooser`：用于选择文件保存或读取的位置。
 - `JOptionPane`：用于弹出消息框，向用户提示保存或读取结果。
- `java.io`：用于文件的输入和输出操作。
 - `FileOutputStream` / `FileInputStream`：用于文件的字节流输出和输入。
 - `ObjectOutputStream` / `ObjectInputStream`：用于将对象序列化并保存到文件，或从文件中读取对象。

2. 主要使用的函数

- `JFileChooser.showSaveDialog()` 和 `JFileChooser.showOpenDialog()`：用于显示保存和读取文件的对话框，允许用户选择文件保存或读取。
- `ObjectOutputStream.writeObject()`：用于将对象序列化并写入到文件中，保存游戏状态。
- `ObjectInputStream.readObject()`：用于从文件中反序列化对象，以恢复游戏状态。

3 计时

`java.util.Timer`：

- `Timer` 类：用于调度执行定时任务。
- `TimerTask` 类：定义需要在指定时间间隔内重复执行的任务。

`java.swing.JOptionPane`：

- `showMessageDialog()`：用于在游戏过程中弹出消息框，通知玩家如超时等事件。

主要函数：

- `Timer.scheduleAtFixedRate()`：用于安排在固定的间隔时间内执行任务。
- `Timer.cancel()`：用于取消当前的定时任务。

4 人机对战

Minimax 算法：

Minimax 是一种递归算法，主要用于两人对抗的博弈问题中。它假设双方都尽力使自己得到最好的结果：

- **基本思想**：Minimax 算法模拟了游戏中所有可能的走法，生成一个决策树。AI 假设自己是最大化玩家，而对手是最小化玩家。
- **过程**：
 1. **最大化和最小化**：AI 在每一步选择可以让自己利益最大化的走法，同时假设对手也总是选择可以让 AI 损失最大的走法。
 2. **递归求解**：通过递归求解，AI 找到对自己最有利的路径。叶子节点（终局）通常用启发式函数进行估值。

Minimax 算法伪代码

```
function Minimax(node, depth, maximizingPlayer):
    if depth == 0 or node is a terminal node:
        return the heuristic value of node

    if maximizingPlayer:
        maxEval = -∞
        for each child of node:
            eval = Minimax(child, depth - 1, false)
            maxEval = max(maxEval, eval)
        return maxEval
```

```

else:
    minEval = +∞
    for each child of node:
        eval = Minimax(child, depth - 1, true)
        minEval = min(minEval, eval)
    return minEval

```

Alpha-Beta 剪枝算法：

Alpha-Beta 剪枝是对 Minimax 算法的一种优化，可以有效减少需要评估的节点数，从而提高效率：

- **基本思想：** Alpha-Beta 剪枝的目标是在 Minimax 搜索过程中，跳过那些不影响最终决策的分支，从而减少计算量。
- **参数定义：**
 - **alpha**：记录当前最大化玩家可以确保的最小收益（初始值为负无穷）。
 - **beta**：记录当前最小化玩家可以确保的最大收益（初始值为正无穷）。
- **剪枝过程：**
 1. 在遍历决策树时，如果当前节点的值大于等于 **beta**（对手可以避免的损失），那么当前分支不可能影响对手的选择，可以直接跳过，这称为 **Beta 剪枝**。
 2. 如果当前节点的值小于等于 **alpha**（自己无法确保的收益），则可以跳过该分支的评估，这称为 **Alpha 剪枝**。
- **优点：** 通过剪枝，可以显著减少需要遍历的节点数，使得搜索速度更快，尤其是在较深的决策树中。

Alpha-Beta 剪枝算法伪代码

```

function AlphaBeta(node, depth, alpha, beta, maximizingPlayer):
    if depth == 0 or node is a terminal node:
        return the heuristic value of node

    if maximizingPlayer:
        maxEval = -∞
        for each child of node:
            eval = AlphaBeta(child, depth - 1, alpha, beta, false)
            maxEval = max(maxEval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break # Beta 剪枝
        return maxEval

    else:
        minEval = +∞
        for each child of node:
            eval = AlphaBeta(child, depth - 1, alpha, beta, true)
            minEval = min(minEval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break # Alpha 剪枝
        return minEval

```

蒙特卡洛树搜索 (MCTS) :

蒙特卡洛树搜索 (MCTS) 是一种基于随机模拟的搜索算法，广泛应用于棋类游戏和对策优化中。它通过模拟对局来评估每一个可能的走法，从而逐步建立决策树并找到最优策略。

基本步骤

MCTS 通常由以下四个步骤组成：

1. 选择 (Selection) :

- 从根节点开始，根据某种策略（如上限置信区间，UCB）选择一个子节点，逐步深入树中，直到到达一个未完全扩展的节点。

2. 扩展 (Expansion) :

- 在选择的节点上扩展一个或多个新节点，表示新的可能走法。

3. 模拟 (Simulation) :

- 从新扩展的节点出发，通过随机模拟一系列走法直至游戏结束，并评估结果。

4. 反向传播 (Backpropagation) :

- 将模拟结果沿着路径反向传播到根节点，更新经过节点的胜负统计信息。

算法的优点

- **无需复杂评估函数**：MCTS 通过大量的模拟自然而然地找到较优解，因此不需要设计复杂的启发式评估函数。
- **适应复杂局面**：由于它的探索机制，MCTS 在面对未知和复杂的博弈局面时特别有效。

MCTS 伪代码

```
function MCTS(root):
    for iteration in 1 to N do:
        node = Selection(root)
        if node is not fully expanded:
            child = Expansion(node)
        else:
            child = node
        result = Simulation(child)
        Backpropagation(child, result)
    return BestChild(root)

function Selection(node):
    while node is fully expanded and not terminal:
        node = BestChild(node, UCB1)
    return node

function Expansion(node):
    add a new child to node for each legal move
    return one of the new children

function Simulation(node):
    while node is not terminal:
        choose a move randomly
        apply the move to the node
```

```
return the result of the game (win/lose/draw)

function Backpropagation(node, result):
    while node is not null:
        update node's statistics with result
        node = node's parent

function BestChild(node, strategy):
    return the child of node with the best score based on the given strategy
    (e.g., UCB1)
```

启发式评估函数：

启发式评估函数（Heuristic Evaluation Function）是用来对某个游戏状态进行评分的函数，通常用于对抗性游戏的 AI 算法中，比如棋类游戏。它的目的是在搜索算法无法达到游戏结束状态时，通过对当前局面的估值来判断局势的优劣，帮助 AI 做出更好的决策。

启发式评估函数的重要性

1. **指导搜索算法：**启发式评估函数可以用来指导搜索算法的决策，特别是在 `Minimax` 或 `Alpha-Beta 剪枝` 等算法中，评估函数可以为非终止节点提供一个估计值，使得算法能够在搜索深度受限时仍然做出较为合理的决策。
2. **减少计算复杂度：**在许多游戏中，搜索整个决策树的时间开销非常大，而启发式评估函数可以通过局面估值，在一定搜索深度下选择相对最优的走法，显著降低计算复杂度。
3. **提高算法的效率和表现：**结合启发式评估函数，可以避免对所有可能的状态进行搜索，而是重点评估更有前景的状态。通过使用合适的启发式策略，AI 可以做出更加接近最优的决策，从而在有限时间内提高表现。

启发式评估函数的常见设计

- **棋子数量：**简单的启发式函数可以通过比较当前棋盘上黑白棋子的数量来进行评估，棋子更多的一方被认为具有优势。
- **棋子位置权重：**在黑白棋中，不同位置的棋子有不同的战略价值，比如角落位置很重要，因此可以为不同位置的棋子赋予不同的权重。
- **行动力：**评估每一方可行的行动数量，即可以落子的合法位置数量。通常拥有更多可行走步的玩家被认为具有优势。
- **边界控制：**对棋盘边缘控制的数量也可能被纳入评估函数中，因为边缘的棋子不容易被对方翻转。

难度调整：

通过控制搜索深度或调整评估函数的复杂度来设置不同难度的AI对手，以适应不同水平的玩家。

AlphaZero

可以导入网上开源的基于AlphaZero的黑白棋模型，但是要注意性能问题。

5 联网控制

使用的 Java 库

1. `java.net`:

- `ServerSocket`: 用于在服务器端创建一个套接字, 监听客户端的连接请求。
- `Socket`: 用于创建客户端和服务器之间的连接, 以便双方可以进行数据传输。

2. `java.io`:

- `ObjectOutputStream` / `ObjectInputStream`: 用于在服务器和客户端之间传递复杂的对象 (例如游戏状态), 实现对象的序列化和反序列化。
- `FileOutputStream` / `FileInputStream`: 用于读取和写入流, 支持对象在文件中的保存。

3. `java.util`:

- `List` 和 `ArrayList`: 用于存储和管理多个客户端的输出流, 以便向所有连接的客户端广播消息。
- `Arrays`: 用于处理数组 (例如深拷贝棋盘)。

主要使用的库函数

- `ServerSocket.accept()`: 等待并接受客户端的连接请求。
- `Socket.getOutputStream()` / `Socket.getInputStream()`: 获取与客户端连接的输入和输出流, 用于通信。
- `ObjectOutputStream.writeObject()` / `ObjectInputStream.readObject()`: 用于发送和接收对象, 支持序列化的对象在客户端和服务器之间传输。
- `Thread.start()`: 为每个客户端启动新的线程来处理它的连接, 以实现并发处理。